

Module 6 Lab Session 2: Scale and Cross-Cutting

Module	M6 Building RESTful Web APIs
Session	2 of 3
Exercises	4 (Pagination, Filtering, and the Audit Filter)

Prerequisites Check Before You Start

Your TMS API must already have:

- CoursesController with GET `/api/courses/{id:int}` and POST `/api/courses` returning the right status codes (Session 1 Exercises 1 + 3).
- CourseResponseDto projected at the query layer with `AsNoTracking()` (Session 1 Exercise 2).
- EnrollmentsController with the nested route `api/courses/{courseId:int}/enrollments` and the 404 → 409 ordering (Session 1 Exercise 3).
- ICourseService and IEnrollmentService registered scoped in `Program.cs`.

If any item is missing, finish Session 1 first. Pagination is a service-layer change on top of a working CourseService, not a parallel rewrite.

What This Session Is Really About

Session 1 built the contract for a single resource. Today the contract has to survive load.

The lab opens with one demo: GET `/api/courses` returning every row in the Courses table at once. With the M5 seed of three or four courses it is fine. With the deterministic seed you add at the start of this session twenty-five courses, all uniquely coded the response is still small, but the **architecture** of the call is wrong: there is no upper bound on what the database returns. Production data has a way of finding the largest `SELECT *` in the codebase.

So today you build the bounded version. One controller action, one service method, one supporting filter, one seeder:

- **PagedRequest** the input contract. Page number, page size with a hard cap, optional search term, optional ordering.

- **PagedResponse<T>** the output contract. Items plus the metadata Angular needs to render pagination controls without a second round-trip.
- **GetCoursesAsync** the service method that ties it all together: filter, count, sort, page, project. The order is the lesson.
- **AuditLogFilter** a registered global action filter that logs every API call and its status code. Cross-cutting, not business.
- **DataSeeder** a deterministic seed of twenty-five courses so verification is reproducible across cohorts.

By the end of this session, GET /api/courses?page=2&pageSize=10&search=fund returns ten or fewer items, the right totalCount, the right totalPages, and a single SQL statement (or two one count, one page) in the EF log. That, not the response body, is the proof.

Before You Begin Add the Deterministic Seeder

Pagination verification is impossible without data. M5's seed is too small (three or four courses) to demonstrate paging boundaries; manual POST to twenty-five courses through Scalar is a forty-five-minute scavenger hunt that no production team would tolerate. Real .NET teams ship a seeder. So will you.

1. Create Data/DataSeeder.cs in your API project

```
using Microsoft.EntityFrameworkCore;
```

```
namespace Tms.Api.Persistence;
```

```
public static class DataSeeder
{
    private static readonly (string Code, string Title, int MaxCapacity)
    [] Courses =
    [
        ("CSE-101", "Web Development Fundamentals", 30),
        ("CSE-102", "TypeScript Essentials", 30),
        ("CSE-103", "Git and Collaborative Workflows", 25),
        ("CSE-201", "ASP.NET Core Fundamentals", 28),
        ("CSE-202", "Entity Framework Core and PostgreSQL", 28),
        ("CSE-203", "Building RESTful Web APIs", 28),
        ("CSE-301", "Advanced Web API Patterns", 24),
        ("CSE-302", "Angular Fundamentals", 26),
        ("CSE-303", "Angular Advanced", 24),
        ("CSE-304", "Full-Stack Integration", 22),
        ("CSE-305", "Testing and Quality Assurance", 22),
        ("CSE-306", "Security and Authentication", 20),
    ]
}
```

```

        ("DAT-101", "Database Design Foundations", 30),
        ("DAT-201", "Advanced SQL and Indexing", 26),
        ("DAT-202", "Data Modelling for the Web", 26),
        ("ARC-101", "Software Architecture Patterns", 22),
        ("ARC-201", "Cloud-Native Architecture", 22),
        ("DEV-101", "DevOps Foundations", 24),
        ("DEV-201", "Continuous Delivery Pipelines", 22),
        ("MOB-101", "Mobile App Foundations", 24),
        ("MOB-201", "Cross-Platform Mobile", 22),
        ("AI-101", "Applied Machine Learning", 20),
        ("AI-201", "Generative AI for Developers", 18),
        ("UX-101", "UX Research and Wireframing", 24),
        ("UX-201", "Design Systems and Tokens", 22),
    ];

    public static async Task SeedAsync(TmsDbContext context, Cancellation
onToken ct = default)
    {
        await context.Database.MigrateAsync(ct);

        if (await context.Courses.AnyAsync(ct))
        {
            return;
        }

        foreach (var (code, title, maxCapacity) in Courses)
        {
            context.Courses.Add(new Course
            {
                Code = code,
                Title = title,
                MaxCapacity = maxCapacity
            });
        }

        await context.SaveChangesAsync(ct);
    }
}

```

Three production habits in this seeder. First, the data is **deterministic** the same twenty-five rows on every cohort, every machine. Random data with Bogus looks impressive but turns lab verification into a guessing game ("does step 4 expect 47 or 52 results?"). Second, the seeder is **idempotent** running it twice does not double-insert. Third, the seeder calls `MigrateAsync` first; on a fresh dev database, schema and data come up in one call.

2. Run the seeder on application startup, only in Development

Open Program.cs. Just before `app.Run();`, add:

```
if (app.Environment.IsDevelopment())
{
    using var scope = app.Services.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<TmsDbContext>();
    await DataSeeder.SeedAsync(context);
}
```

The `IsDevelopment()` gate matters. You do **not** want this code to run in production staging and production data is owned by the operations team, not the seed file. The `using var scope = ...` pattern is the recommended way to resolve a scoped service (`TmsDbContext` is scoped) outside of a request without it, you would resolve the context from the root provider and leak a connection.

3. Verify the seed

Run the API. Confirm in pgAdmin (or psql):

```
SELECT COUNT(*) FROM "Courses";
```

Expected: 25. If you already had courses from Session 1's manual POST requests, the seeder skipped the seed (the idempotency guard). Truncate the table or drop the database and re-run if you want a clean baseline:

```
TRUNCATE "Courses" RESTART IDENTITY CASCADE;
```

(Adjust if your migration uses a different schema or table name. `RESTART IDENTITY` resets the auto-increment so seeded IDs start at 1 again.)

Checkpoint: `SELECT COUNT(*) FROM "Courses";` returns 25 (or your Session 1 manual rows + 25 if you did not truncate). The API still starts cleanly.

Exercise 4: Pagination, Filtering, and the Audit Filter (LO 6.4)

Scenario: Yared opens the dashboard and runs `GET /api/courses`. Today it returns twenty-five rows; in three semesters it will return five hundred. The browser will not freeze yet, but the **architecture** is wrong every page-load downloads the entire table. You make pagination mandatory, with a capped page size, a stable sort, and a `PagedResponse<T>` shape Angular can render without a second round-trip. Then you register a global `AuditLogFilter` so every API call is

logged with its status code, and the cross-cutting concern stays out of every controller action.

This exercise has four parts. Do them in order Part B depends on Part A's types, Part C depends on Part B's controller action, Part D is independent but easier once the paginated path is producing log lines.

Part A Define the pagination contracts

Create Dtos/PagedRequest.cs:

```
namespace Tms.Api.Dtos;

public record PagedRequest
{
    private const int MaxPageSize = 50;
    private int _pageSize = 20;

    public int Page { get; init; } = 1;

    public int PageSize
    {
        get => _pageSize;
        init => _pageSize = value < 1 ? 20 : value > MaxPageSize ? MaxP
ageSize : value;
    }

    public string? Search { get; init; }
    public string OrderBy { get; init; } = "Title";
    public bool Descending { get; init; }
}
```

Three details that earn their place in production code. The `MaxPageSize` constant is a single source of truth never inlined as a magic 50. The `PageSize` setter clamps both ends: a hostile client sending `?pageSize=10000` lands on 50; a confused client sending `?pageSize=0` lands on the default 20. And `OrderBy` is a plain string with a default Part C decides which subset of `OrderBy` values is actually safe to translate into LINQ (you do **not** want clients smuggling arbitrary strings into your query).

Create Dtos/PagedResponse.cs:

```
namespace Tms.Api.Dtos;

public record PagedResponse<T>
{
    public required IReadOnlyList<T> Items { get; init; }
```

```

    public required int TotalCount { get; init; }
    public required int Page { get; init; }
    public required int PageSize { get; init; }
    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);
    public bool HasPrevious => Page > 1;
    public bool HasNext => Page < TotalPages;
}

```

TotalPages, HasPrevious, HasNext are computed properties the wire format will include them in the JSON because they are public getters, but the writer never has to set them. The Angular pagination control reads hasNext and hasPrevious to enable or disable the next/previous buttons, and reads totalPages to render Page 2 of 5. All in one response.

Part B Add the paginated GET action

Open Controllers/CoursesController.cs. The new action sits next to GetCourseById. Both can coexist because attribute routing distinguishes [HttpGet] (collection) from [HttpGet("{id:int}", ...)] (single item):

```

[HttpGet]
public async Task<IActionResult> GetCourses(
    [FromQuery] PagedRequest request, CancellationToken ct)
{
    var result = await courseService.GetCoursesAsync(request, ct);
    return Ok(result);
}

```

[FromQuery] binds the request from query-string parameters ?page=2&pageSize=10&search=fund becomes a populated PagedRequest. Without [FromQuery], ASP.NET Core would try to bind from the body, which is wrong for a GET.

Part C Implement GetCoursesAsync in the service

Extend ICourseService.cs:

```

public interface ICourseService
{
    Task<CourseResponseDto?> GetByIdAsync(int id, CancellationToken ct);

    Task<CourseResponseDto> CreateAsync(CreateCourseRequest request, CancellationToken ct);

    Task<bool> CodeExistsAsync(string code, CancellationToken ct);
}

```

```

        Task<PagedResponse<CourseResponseDto>> GetCoursesAsync(PagedRequest
request, CancellationToken ct);
}

```

In CourseService.cs, this is the **graded** part of the lab the order of LINQ operations is exactly what the integrity-lab Fake Pagination deduction (-5 if TotalCount is computed after Skip/Take) inspects. The hints below tell you what each step does; the assembly is yours:

```

public async Task<PagedResponse<CourseResponseDto>> GetCoursesAsync(
    PagedRequest request, CancellationToken ct)
{
    // TODO 1: Start with a no-tracking IQueryable<Course>:
    //     IQueryable<Course> query = context.Courses.AsNoTracking();

    // TODO 2: If request.Search has a value, append a Where clause:
    //     query = query.Where(c => EF.Functions.ILike(c.Title, $"{request.Search}%"))
    //                                     || EF.Functions.ILike(c.Code, $"{request.Search}%"));
    //     ILike is the case-insensitive LIKE in PostgreSQL using it here means
    //     the search "fund" finds "Web Development Fundamentals" without
    //     learners
    //     being surprised by case-sensitivity at Lab time.

    // TODO 3: Count BEFORE paging:
    //     var totalCount = await query.CountAsync(ct);
    //     This produces one SELECT COUNT(*) statement. If you Count after
    //     Skip/Take,
    //     you would get the count of the page, not the total.

    // TODO 4: Apply OrderBy, then Skip/Take, then Select projection.
    //     For OrderBy, branch on request.OrderBy ∈ { "Title", "Code", "MaxCapacity" }
    //     and apply Descending if request.Descending. Reject unknown OrderBy values
    //     silently by falling back to "Title" never let an arbitrary string
    //     into the LINQ tree.

    // TODO 5: Materialise:
    //     var items = await sortedQuery
    //         .Skip((request.Page - 1) * request.PageSize)
    //         .Take(request.PageSize)
    //         .Select(c => new CourseResponseDto(c.Id, c.Code, c.Title,
    //             c.MaxCapacity, c.Enrollments.Count))
    //         .ToListAsync(ct);
}

```

```
// TODO 6: Return new PagedResponse<CourseResponseDto> { Items = items, TotalCount = totalCount, Page = request.Page, PageSize = request.PageSize };
```

```
    throw new NotImplementedException();
}
```

A note on the SQL log: with logging on, you should see exactly two statements per request one `SELECT COUNT(*)` and one `SELECT ... LIMIT 10 OFFSET 10` (or whatever your `PageSize` and `Page` produce). If you see twenty-six statements, the `Select` projection is happening in C# memory instead of SQL confirm the projection is **inside** the `IQueryable` chain and not after a `.ToListAsync()`.

Part D Add the global audit filter

Cross-cutting concerns live in filters, not in every action. The audit log line “TMS API call: GET /api/courses → 200” is the smallest possible breadcrumb a production operations team needs to answer the question “did the dashboard try to call us at 14:03?”.

Create `Filters/AuditLogFilter.cs`:

```
using Microsoft.AspNetCore.Mvc.Filters;

namespace Tms.Api.Filters;

public class AuditLogFilter(ILogger<AuditLogFilter> logger) : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        var route = context.HttpContext.Request.Path;
        var method = context.HttpContext.Request.Method;
        logger.LogInformation("TMS API call: {Method} {Route}", method, route);
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        var status = context.HttpContext.Response.StatusCode;
        logger.LogInformation("TMS API response: {StatusCode}", status);
    }
}
```

Register it globally in `Program.cs`:


```
builder.Services.AddControllers(options =>
{
    options.Filters.Add<AuditLogFilter>();
});
```

Add<T>() (the generic overload) registers the filter type so DI can resolve ILogger<AuditLogFilter> for you. Add(new AuditLogFilter(...)) would force you to construct the logger by hand unnecessary.

The boundary that matters in the deck and in interviews: filters are for **cross-cutting** concerns. If you find yourself writing if (course.IsFull) inside a filter, that decision belongs in EnrollmentService the M6 Topic 6 Socratic question is "what goes wrong if an action filter starts checking 'is this course full?'" The answer: the filter becomes part of the business contract, gets duplicated as EnrollmentService evolves, and breaks the moment two calls disagree.

Step 5 Verify

Step	Action	Expected status	Expected body / observation
1	GET /api/courses? page=1&pageSi ze=10	200 OK	items.length == 10, totalCount == 25, totalPages == 3, hasNext == true, hasPrevious == false
2	GET /api/courses? page=3&pageSi ze=10	200 OK	items.length == 5, totalCount == 25, hasNext == false, hasPrevious == true
3	GET /api/courses? pageSize=9999	200 OK	pageSize == 50 in the response the cap kicked in
4	GET /api/courses? search=fund	200 OK	One item: CSE-101 Web Development Fundamentals. totalCount == 1. (fund matches Fundamentals case-insensitively.)
5	GET /api/courses? orderBy=Code& descending=true&pageSize=5	200 OK	First item is UX-201 (alphabetically last by code), descending order
6	Tail the API console while step 1 runs	(logs)	Two log lines per request: TMS API call: GET /api/courses and TMS API response: 200
7	Tail EF Core SQL logs while	(SQL)	One SELECT COUNT(*) FROM "Courses" and one SELECT ... LIMIT 10 OFFSET 0

Step	Action	Expected status	Expected body / observation
	step 1 runs (M5 logging habit)		exactly two statements

For step 4, if your local PostgreSQL collation is case-sensitive and the search returns zero results, the `EF.Functions.ILike` call recommended in TODO 2 fixes it. `Like` is case-sensitive on PostgreSQL by default, `ILike` is the case-insensitive sibling. If your team is on SQL Server, the equivalent is `EF.Functions.Like(c.Title, $"%{request.Search}%")` which is collation-dependent (usually case-insensitive).

For step 7, if you see twenty-six statements, the `Select` projection is happening in memory re-read the order: `IQueryable` → `Where` → `CountAsync` → `OrderBy` → `Skip` → `Take` → `Select` → `ToListAsync`. The projection must stay inside the `IQueryable` chain.

Troubleshooting

Symptom	Likely cause	Fix
<code>totalCount</code> equals <code>items.Count</code> instead of the table total	<code>CountAsync</code> called after <code>Skip/Take</code>	Count first, then skip/take/project order matters. This is the Fake Pagination deduction in the integrity lab.
<code>pageSize=9999</code> returns 9999 items	Property setter cap missing	Confirm <code>init => _pageSize = value < 1 ? 20 : value > MaxPageSize ? MaxPageSize : value;</code>
<code>?search=fund</code> returns nothing	PostgreSQL collation is case-sensitive; <code>Like</code> is case-sensitive	Switch to <code>EF.Functions.ILike(...)</code> (PostgreSQL-specific, case-insensitive)
<code>?orderBy=DROP TABLE</code> produces a runtime exception	<code>OrderBy</code> string passed straight into a dynamic LINQ helper	Whitelist the allowed columns (<code>Title</code> , <code>Code</code> , <code>MaxCapacity</code>); fall back to <code>Title</code> for anything else
Two routes both match <code>GET /api/courses/5</code>	Both actions decorated with bare <code>[HttpGet]</code>	Single-item action must be <code>[HttpGet("{id:int}", Name = nameof(GetCourseById))]</code>
Audit filter logs	Filter applied to	Filters only run on requests that reach a

Symptom	Likely cause	Fix
every static-file request	non-controller endpoints (it should not)	controller action if you see static-file logs, you wired this as middleware, not a filter
EF logs show 26 statements per page request	Projection runs after ToListAsync	Move the <code>Select(c => new CourseResponseDto(...))</code> before <code>ToListAsync(ct)</code>

Why this session ends here

The dashboard now scales:

- **Collections are bounded by default.** A new endpoint added next semester that forgets pagination is a code-review failure, not a production incident because every existing collection action takes `[FromQuery] PagedRequest`, the team eye learns to flag the absence.
- **The page-size cap is server-controlled, not client-honour-system.** `?pageSize=10000` lands on fifty. The Angular team can request whatever feels right; the server protects itself.
- **The order of LINQ operations is the lesson, not an implementation detail.** Filter, count, sort, skip/take, project. Every senior interview asks at least one variant of this and now you can answer it with the file path of your own service method as proof.
- **Cross-cutting logic stays cross-cutting.** Every API call produces two log lines, regardless of which controller handled it. Add a thirty-first controller next quarter it logs without anyone editing the seeder, the controller, or the filter.

You did **not** add HATEOAS links to the paginated response. That is deliberate. Links belong on the detail endpoint where a client picks one resource and asks "what can I do with this?" a list of fifty courses with a links array per item is noise. Session 3 adds the links to `GET /api/courses/{id}` and enriches the Scalar metadata so the documentation page becomes a usable contract.

Session 2 Checkpoint

Before moving to Session 3, confirm all six rows pass:

Check	Pass means
GET /api/courses?page=1&pageSize=10	items.length == 10, totalCount == 25, totalPages == 3
GET /api/courses?pageSize=9999	Response pageSize == 50 (capped)
GET /api/courses?search=fund	One item, totalCount == 1
EF SQL log for any paginated GET	Exactly one SELECT COUNT(*) and one paged SELECT ... LIMIT ... OFFSET ...
Console log for any API call	Two lines: TMS API call: METHOD route and TMS API response: STATUS
GET /api/courses/1 (single item)	Still returns the Session 1 CourseResponseDto the new collection action did not break the existing single-item action

If any row fails, fix it before Session 3. HATEOAS links and Scalar metadata sit on top of a paginated, audited collection they do not paper over a broken one.